HAFTA 4 • CUMARTESİ

Transformer & BERT

Hugging Face Pipeline & Fine-Tuning

AutoTokenizer · AutoModel · pipeline() · DistilBERT · Trainer API · datasets · evaluate

110M	7 Saat	4 Dosya	800+
BERT-base Parametre	Net Eğitim Süresi	Python Uygulaması	Satır Kod

09:00 — 17:00 · Teoriden pratiğe · GPU opsiyonel

Günün Planı — Hafta 4 Cumartesi

09:00–09:30
Geçen Hafta Özeti & Giriş

09:30–10:45
Transformer & BERT Mimarisi

10:45–11:00
Kısa Mola

11:00–12:00
Hugging Face Ekosistemi

12:00–13:00
Tokenizer & Datasets

13:00–14:00
Öğle Arası

14:00–15:30
Fine-Tuning Trainer API

15:30–16:30
Değerlendirme & Metrikler

16:30–17:00
Ödev & Kapanış

Gün Sonu Hedefi: pipeline() ile 5 NLP görevi çalıştır → DistilBERT'i IMDB'ye fine-tune et → F1/AUC hesapla → modeli HF Hub'a kaydet

🤗 Hugging Face Ekosistemi

Model Hub · transformers · datasets · evaluate · PEFT · Spaces

transformers

AutoModel, pipeline,
Trainer API, 200K+ model

datasets

load_dataset, map,
60K+ veri seti, Arrow

evaluate

BLEU, F1, AUC,
seqeval, accuracy

Hugging Face

PEFT

LoRA, QLoRA,
Prefix Tuning, IA3

Inference API

Bulut API çıkarımı,
HF Hub endpoint

tokenizers

Rust tabanlı hızlı
tokenizer (Rust FFI)

```
Model Hub: 900K+ model · 300K+ veri seti · 150K+ Space · pip install transformers datasets evaluate peft accelerate
```

Transformer Mimarisi — Üç Varyant

Encoder-Only · Decoder-Only · Encoder-Decoder | Self-Attention · Multi-Head · Add & LayerNorm

Encoder-Only

Modeller

BERT · RoBERTa
DistilBERT · ALBERT

NLP Görevleri

Metin sınıflandırma
NER · Soru cevap
Anlam benzerliği

Dikkat Türü

Çift yönlü — tüm tokenlara bakabilir

Decoder-Only

Modeller

GPT-2 · GPT-3/4
LLaMA · Mistral

NLP Görevleri

Metin üretimi
Kod üretimi
Sohbet modeli

Dikkat Türü

Tek yönlü — sadece önceki tokenlara

Encoder-Decoder

Modeller

T5 · BART
mT5 · FLAN-T5

NLP Görevleri

Çeviri · Özetleme
Soru üretme
Seq2Seq görevler

Dikkat Türü

Cross-attention: encoder → decoder

$\text{Attention}(Q,K,V) = \text{softmax}(QK^T/\sqrt{d_k})\cdot V$ | Multi-Head: $\text{concat}(\text{head}_1,\ldots\text{head}_h)\cdot W_o$ | Pozisyon: $PE(pos,2i)=\sin(pos/10000^{(2i/d)})$

BERT — Bidirectional Encoder Representations from Transformers

Devlin et al., Google 2018 · MLM + NSP Ön-Eğitim · 110M / 340M Parametre

Ön-Eğitim 1: MLM (Maskeli Dil Modeli)

Token'ların %15'i seçilir:
%80'i → [MASK] ile değiştirilir
%10'u → rastgele token
%10'u → olduğu gibi kalır
Model gizlenen tokeni tahmin eder.

Ön-Eğitim 2: NSP (Sonraki Cümle Tahmini)

İki cümle çifti verilir:
%50 → gerçek sonraki cümle (IsNext)
%50 → rastgele cümle (NotNext)
Not: RoBERTa NSP'yi kaldırdı, daha iyi.

Girdi Temsili (3 Embedding Toplamı)

[CLS] + Cümle A + [SEP] + Cümle B + [SEP]
Token Embedding (kelime)
+ Segment Embedding (A veya B)
+ Positional Embedding (öğrenilmiş)

BERT Ailesi

BERT-base	12 katman · 768 boyut · 110M param
BERT-large	24 katman · 1024 boyut · 340M param
DistilBERT	6 katman · 66M param · %97 performans
RoBERTa	NSP yok · daha büyük veri · daha iyi

Metin Sınıflandırma

`BertForSequenceClassification`

Token Sınıflandırma (NER)

`BertForTokenClassification`

Soru Cevaplama (QA)

`BertForQuestionAnswering`

Anlam Benzerliği

`BertModel + cosine sim.`

CLS → cümle temsili · SEP → cümle sınırı · WordPiece: 'playing' → ['play','##ing'] · Vocab: 30.522 token

Hugging Face pipeline() — Beş Satırda NLP

AutoModelForXxx · AutoTokenizer · device=0 (GPU) · batch_size · aggregation_strategy

pipeline_ornekler.py

```
from transformers import pipeline

# 1. Duygu Analizi
sentiment = pipeline('sentiment-analysis')
r = sentiment('Bu film harikaydı!')
# → [{'label': 'POSITIVE', 'score': 0.9998}]

# 2. Sıfır-Atış Sınıflandırma
zs = pipeline('zero-shot-classification',
    model='facebook/bart-large-mnli')
r = zs('Bugün maç var!',
    candidate_labels=['spor','siyaset','bilim'])

# 3. NER — Varlık Tanıma
ner = pipeline('ner', model='dslim/bert-base-NER',
    aggregation_strategy='simple')
r = ner('Elon Musk Tesla\'da CEO\'dur.')
# → [{'entity':'PER','word':'Elon Musk',...}]

# 4. Soru-Cevap (Extractive)
qa = pipeline('question-answering',
    model='deepset/roberta-base-squad2')
r = qa(question='Kim kurdu?',
    context='SpaceX\'ı Elon Musk 2002\'de kurdu.')
```

Duygu Analizi

distilbert-base-sst-2

Zero-Shot Sınıf.

facebook/bart-large-mnli

Varlık Tanıma (NER)

dslim/bert-base-NER

Soru-Cevap

deepset/roberta-base-squad2

Metin Üretimi

gpt2

Batch: pipeline()(texts, batch_size=32) · GPU: device=0 · Özel: pipeline(task, model='kullanici/model-adi')

```
gen = pipeline('text-generation', model='gpt2', device=0)
out = gen('Yapay zeka', max_new_tokens=50
```

Tokenizer — Metinden Sayısal Tensöre

WordPiece · BPE · SentencePiece · Offset Mapping · Özel Tokenlar · Padding

1. Ham Metin	2. Normalize	3. Pre-tok.	4. Alt-kelime	5. Özel Tok.	6. ID'ye Çevir
'Transformers kütüphanesi'	Lowercase, Unicode	Boşluk bölme	'transform', '##ers',...	[CLS]+tok +[SEP]	[101, 2237, ..., 102]

tokenizer_kullanimi.py

```
from transformers import AutoTokenizer
tok = AutoTokenizer.from_pretrained('bert-base-uncased')

enc = tok('Transformers harika!',
    return_tensors='pt',        # 'pt'/'tf'/'np'
    padding='max_length',       # sabit uzunluk
    truncation=True,
    max_length=128,
    return_offsets_mapping=True, # karakter pozisyon
)
# enc → input_ids, attention_mask, token_type_ids
tok.convert_ids_to_tokens(enc['input_ids'][0])
# ['[CLS]','transform','##ers','fantastic','!','[SEP]']
```

[CLS]	Sınıflandırma için cümle temsili. Her girdinin başına eklenir.
[SEP]	Cümle sınırı / cümleler arası ayraç.
[PAD]	attention_mask=0. Model bu tokeni dikkate almaz.
[MASK]	MLM ön-eğitiminde gizlenen token.
[UNK]	Vocab'da bulunmayan karakterler için.

datasets API — Veri Yükleme & Ön İşleme

load_dataset · DatasetDict · map(batched=True) · Arrow Format · Otomatik Cache

datasets_api.py

```
from datasets import load_dataset
from transformers import AutoTokenizer

# ── 1. Yükleme ──────────────────────────────────────
ds = load_dataset('imdb')  # otomatik indir & cache
# DatasetDict({train:Dataset(25000), test:...})
print(ds['train'].features)

# ── 2. Tokenizasyon (batch map) ─────────────────────
tok = AutoTokenizer.from_pretrained('distilbert-base-uncased')
def tokenize(batch):
    return tok(batch['text'], truncation=True, max_length=256)

tok_ds = ds.map(tokenize,
    batched=True,       # 5-10× daha hızlı
    num_proc=4,         # çoklu CPU işlemci
    remove_columns=['text']  # ham sütun sil
)

# ── 3. Format & Stratified Split ────────────────────
tok_ds.set_format('torch')  # 'tensorflow'/'numpy'
split = tok_ds['train'].train_test_split(
    test_size=0.1, seed=42,
    stratify_by_column='label'  # dengeli bölme
)
```

Arrow Format

Bellek eşlemeli (memory-mapped). 100GB+ veri setleri için ideal. Sıfır kopya operasyonları.

Otomatik Cache

~/.cache/huggingface/datasets — map() dönüşümleri cache'lenir. Tekrar çalıştırma gerekmez.

Stratified Split

train_test_split(stratify_by_column='label') — sınıf oranlarını korur.

60.000+ Veri Seti

IMDB, CoNLL-2003, SQuAD, GLUE, OSCAR (Türkçe), C4, Common Crawl...

`ds.filter(lambda x: x['label']==1)` · `ds.shuffle(seed=42)` · `ds.select(range(1000))` · `ds.rename_column()`

```
# ── 4. Manuel DataLoader (opsiyonel) ────────────────
```

Fine-Tuning — Trainer API ile DistilBERT (IMDB)

TrainingArguments · EarlyStopping · compute_metrics · push_to_hub()

distilbert_finetuning.py

```
from transformers import (
    AutoModelForSequenceClassification, AutoTokenizer,
    TrainingArguments, Trainer,
    DataCollatorWithPadding, EarlyStoppingCallback)
import evaluate, numpy as np

# ── 1. Model yükle ──────────────────────────────
model = AutoModelForSequenceClassification.from_pretrained(
    'distilbert-base-uncased', num_labels=2,
    id2label={0:'NEG',1:'POS'},
    label2id={'NEG':0,'POS':1})

# ── 2. TrainingArguments ────────────────────────
args = TrainingArguments(
    output_dir='./distilbert-imdb',
    num_train_epochs=3,
    per_device_train_batch_size=32,
    learning_rate=2e-5,        # küçük LR kritik
    warmup_ratio=0.1,
    weight_decay=0.01,
    evaluation_strategy='epoch',
    load_best_model_at_end=True,
    metric_for_best_model='f1')

# ── 3. Metrik + Trainer + Eğitim ────────────────
def compute_metrics(ep):
    preds = np.argmax(ep.predictions, -1)
```

trainer_run.py

```
trainer = Trainer(model=model, args=args,
    train_dataset=train_ds, eval_dataset=val_ds,
    tokenizer=tokenizer,
    data_collator=DataCollatorWithPadding(tokenizer),
    compute_metrics=compute_metrics,
    callbacks=[EarlyStoppingCallback(patience=2)],
)
trainer.train()
trainer.evaluate()
trainer.save_model('./distilbert-imdb')
trainer.push_to_hub('kullanici/distilbert-imdb')
```

Öğrenme Hızı

2e-5 ~ 5e-5 ideal. Warmup ile başla (warmup_ratio=0.1). Linear/cosine decay.

Batch Boyutu

32-64 daha kararlı. GPU dolmuyorsa gradient_accumulation_steps=4.

Epoch Sayısı

3-5 epoch çoğunlukla yeterli. EarlyStopping ile overfitting önle.

Beklenen: DistilBERT IMDB → acc ≈ 0.92, F1 ≈ 0.92 (3 epoch, 2e-5 LR, batch=32) · BERT-base → ~0.94

Fine-Tuning Stratejileri — Ne Zaman Hangisi?

Feature Extraction · Katman Dondurma · Tam Fine-Tuning · LoRA / QLoRA

1. Feature Extraction ~%84–87

Tüm BERT katmanları dondurulur.
Yalnızca sınıflandırıcı baş eğitilir.
Eğitilebilir: ~1.500 param (2 sınıf)

Ne Zaman?
Çok küçük veri (<500)
Hız öncelikli
BERT yakın domain

2. Son 2 Katman FT ~%88–90

İlk N katman dondurulur.
Son 2-4 blok + baş eğitilir.
Eğitilebilir: ~7M / 66M param

Ne Zaman?
Orta veri (500-5K)
GPU belleği kısıtlı
Domain kayması orta

3. Tam Fine-Tuning ~%92–94

Tüm parametreler güncellenir.
66M (DistilBERT) / 110M (BERT).
AdamW + warmup + weight decay.

Ne Zaman?
Yeterli veri (5K+)
GPU mevcut
En yüksek performans

4. LoRA (PEFT) ~%92–93

Ağırlıklar dondurulur, $\Delta W = A \cdot B$ eklenir.
Param: ~0.1-1% (r=8: ~500K).
QLoRA: 4-bit quant + az bellek.

Ne Zaman?
Büyük modeller (7B+)
Sınırlı GPU belleği
Hızlı denemeler

Öneri: Küçük veri → Feature Extraction · Orta veri → Kısmi FT · Büyük veri + GPU → Tam FT · 7B+ model → LoRA

Bugünkü Uygulamalar — 4 Python Dosyası

pip install transformers datasets evaluate torch scikit-learn matplotlib seaborn

Uygulama 01 — Pipeline & Tokenizer

`uygulama_01_pipeline_tokenizer.py`

- 5 pipeline görevi: duygu, NER, QA, özetleme, üretim
- AutoTokenizer: offset mapping, özel tokenlar, WordPiece
- Batch tokenizasyon hız karşılaştırması (single vs batched)
- Güven skoru & eşik analizi görselleştirmesi
- Tokenizasyon adımları interaktif görsel

Uygulama 02 — datasets API & Veri İşleme

`uygulama_02_datasets.py`

- load_dataset('imdb') → DatasetDict yapısı inceleme
- map(batched=True, num_proc=4) ile toplu tokenizasyon
- Sınıf dengesizliği analizi + stratified split
- Uzunluk dağılımı & özellik istatistikleri
- DataLoader ile manuel batch döngüsü gösterimi

Uygulama 03 — DistilBERT Fine-Tuning (IMDB)

`uygulama_03_finetuning.py`

- Feature Extraction / Kısmi FT / Tam FT ablasyonu
- TrainingArguments + Trainer + EarlyStopping
- compute_metrics: F1, AUC, Accuracy, Precision/Recall
- Öğrenme eğrisi & confusion matrix görselleştirmesi
- push_to_hub() ile HF Hub'a model yükleme

Uygulama 04 — Kapsamlı NLP Değerlendirmesi

`uygulama_04_degerlendirme.py`

- 3 model × 3 strateji performans karşılaştırması
- ROC eğrisi, PR eğrisi, kalibrasyon analizi
- Hata analizi: yanlış sınıflanan örnekler incelemesi
- Eşik optimizasyonu (F1-max threshold seçimi)
- Radar grafiği ile kapsamlı benchmark tablosu

`pip install transformers datasets evaluate torch scikit-learn matplotlib seaborn | GPU opsiyonel — CPU ile de çalışır`

Hafta 4 Cumartesi — Kazanımlar

- ✓ Hugging Face ekosistemi: transformers, datasets, evaluate, PEFT
- ✓ AutoTokenizer: WordPiece, offset mapping, özel tokenlar
- ✓ BERT mimarisi: MLM, NSP, CLS token, segment embedding
- ✓ 4 fine-tuning stratejisi: Feature Ext. → Tam FT → LoRA
- ✓ pipeline() ile 5 NLP görevi — 3-5 satır kod yeterli
- ✓ datasets API: load_dataset, map(batched), Arrow cache
- ✓ Trainer API: TrainingArguments, EarlyStopping, compute_metrics
- ✓ HF Hub: push_to_hub(), model kartı, topluluk paylaşımı

Pazar: Generatif Modeller — VAE ile MNIST latent space görselleştirme + DCGAN ile Fashion-MNIST görüntü üretimi